ANN and DL (CAI-361)

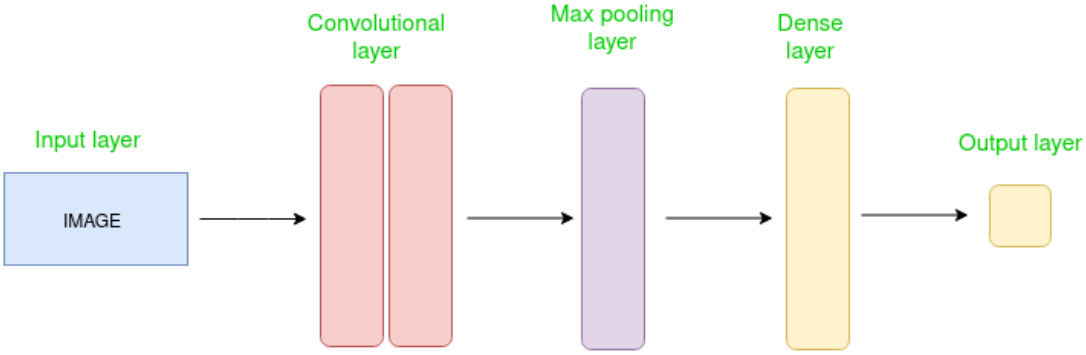
Objectives:

- Convolutional Neural Networks

Convolutional Neural Network:

A **Convolutional Neural Network (ConvNet/CNN)** is a Deep Learning algorithm that can take an image as input, assign importance (learnable weights and biases) to various aspects/objects in the image, and be able to differentiate one from the other. CNNs, are a specialized kind of neural network for processing data that has a known, grid-like topology. Examples include time-series data, which can be thought of as a 1D grid taking samples at regular time intervals, and image data, which can be thought of as a 2D grid of pixels.

A CNN contains following layered architecture.

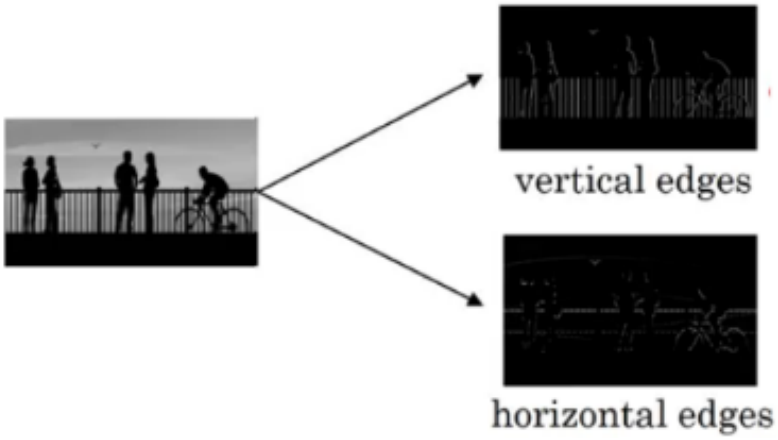


In a convolutional network (ConvNet), there are basically three types of layers:

1. Convolution layer
2. Pooling layer
3. Fully connected layer

Convolutional Layer:

This layer performs a linear operation called **"convolution".** Suppose we are given the below image:



As you can see, there are many vertical and horizontal edges in the image. The first thing to do is to detect these edges. But how do we detect these edges? To illustrate this, let's take a 6 X 6 gray scale image (i.e. only one channel):

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

Next, we convolve this 6×6 matrix with a 3×3 matrix which is called **filter**/ **kernel**/ **feature detector**:

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	**1**	3
0	**1**	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6 X 6 image

*

1	0	-1
1	0	-1
1	0	-1

3 X 3 filter

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

After the convolution, we will get a 4×4 image which is called **feature map**. The first element of the 4×4 matrix will be calculated as follows. We take the first 3×3 matrix from the 6×6 image and multiply it with the filter. Now, the first element of the 4 X 4 output will be the sum of the element-wise product of these values, i.e. 3×1 + 0 + 1×-1 + 1×1 + 5×0 + 8×-1 + 2×1 + 7×0 + 2×-1 = -5. To calculate the second element of the 4 X 4 output, we will shift our filter one step towards the right and again get the sum of the element-wise product. Similarly, we will convolve over the entire image and get a 4×4 output. So, convolving a 6×6 input with a 3×3 filter gave us an output of 4×4.

Consider one more example:

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

*

Note: Higher pixel values represent the brighter portion of the image and the lower pixel values represent the darker portions. This is how we can detect a vertical edge in an image.

More Edge Detection

The type of filter that we choose helps to detect the vertical or horizontal edges. We can use the following filters to detect different edges:

1	0	-1
1	0	-1
1	0	-1

Vertical

1	1	1
0	0	0
-1	-1	-1

Horizontal

Some of the commonly used filters are:

1	0	-1
2	0	-2
1	0	-1

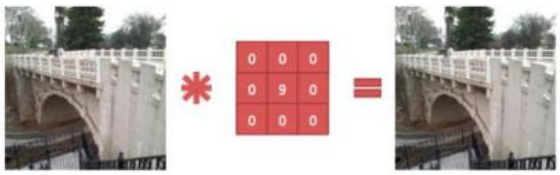
Sobel filter

3	0	-3
10	0	-10
3	0	-3

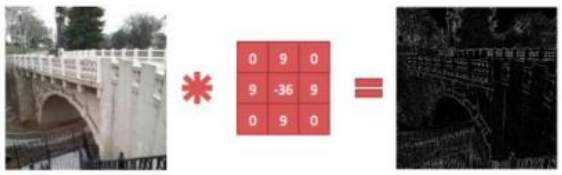
Scharr filter

The Sobel filter puts a little bit more weight on the central pixels. Instead of using these filters, we can create our own as well and treat them as a parameter which the model will learn using back propagation.

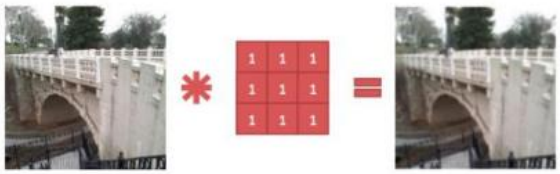
Some more kernels with their effects on images are given below.



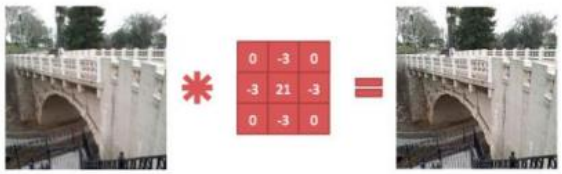
(a) Identity kernel



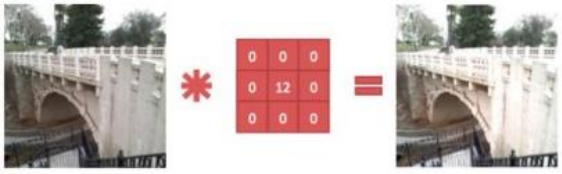
(b) Edge detection kernel



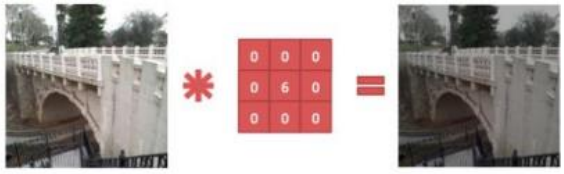
(c) Blur kernel



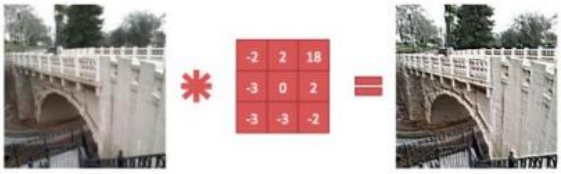
(d) Sharpen kernel



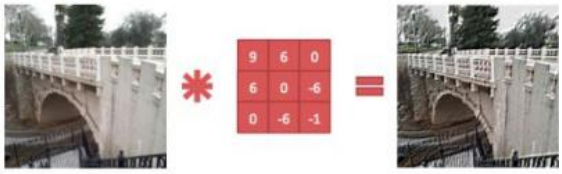
(e) Lighten kernel



(f) Darken kernel



(g) Random kernel 1



(h) Random kernel 2

Padding

We have seen that convolving an input of 6×6 dimension with a 3×3 filter results in 4×4 output. We can generalize it and say that if the input is n×n and the filter size is f×f, then the output size will be (n-f+1) × (n-f+1):

- **Input:** n × n
- **Filter size:** f × f
- **Output:** (n-f+1) × (n-f+1)

There are primarily two disadvantages here:

1. Every time we apply a convolutional operation, the size of the image shrinks
2. Pixels present in the corner of the image are used only a few number of times during convolution as compared to the central pixels. Hence, we do not focus too much on the corners since that can lead to information loss

To overcome these issues, we can pad the image with an additional border, i.e., we add one pixel all around the edges. This means that the input will be an 8×8 matrix (instead of a 6×6 matrix). Applying convolution of 3×3 on it will result in a 6×6 matrix which is the original shape of the image. This is where padding comes to the fore:

- **Input:** n×n
- **Padding:** p
- **Filter size:** f×f
- **Output:** (n+2p-f+1) X (n+2p-f+1)

There are two common choices for padding:

1. **Valid:** It means no padding. If we are using valid padding, the output will be (n-f+1) × (n-f+1)
2. **Same:** Here, we apply padding so that the output size is the same as the input size, i.e.,
   n+2p-f+1 = n
   So, p = (f-1)/2

We now know how to use padded convolution. This way we don't lose a lot of information and the image does not shrink either. Next, we will look at how to implement strided convolutions.

Strided Convolutions

Suppose we choose a stride of 2. So, while convoluting through the image, we will take two steps – both in the horizontal and vertical directions separately. The dimensions for stride *s* will be:

- **Input:** n × n
- **Padding:** p
- **Stride:** s
- **Filter size:** f × f
- **Output:** [(n+2p-f)/s+1] X [(n+2p-f)/s+1]

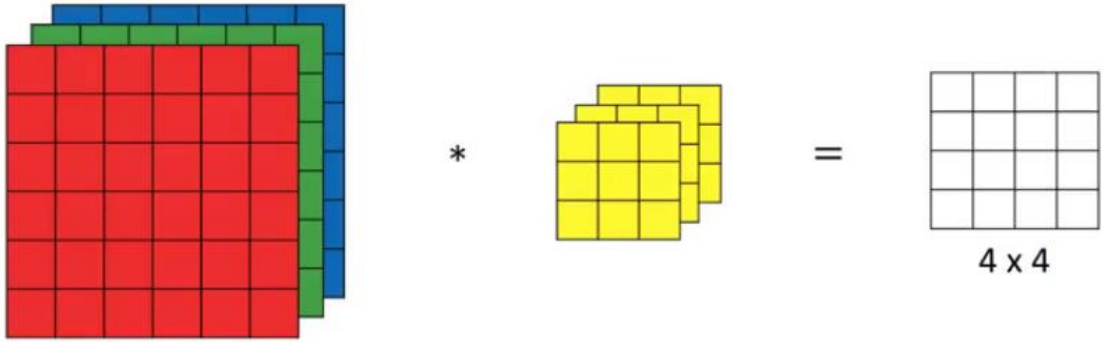
Stride helps to reduce the size of the image, a particularly useful feature.

Convolutions Over Volume

Suppose, instead of a 2-D image, we have a 3-D input image of shape 6×6×3. How will we apply convolution on this image? We will use a 3×3×3 filter instead of a 3×3 filter. Let's look at an example:

- **Input:** 6×6×3
- **Filter:** 3×3×3

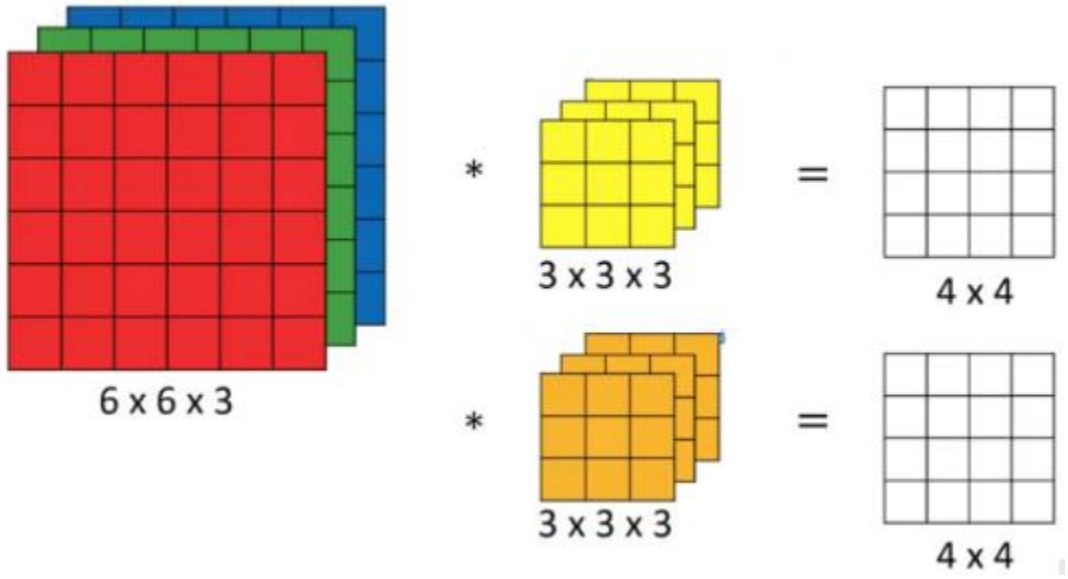
The dimensions above represent the height, width and channels in the input and filter. ***Keep in mind that the number of channels in the input and filter should be same.*** This will result in an output of 4×4. Let's understand it visually:



Since there are three channels in the input, the filter will consequently also have three channels. After convolution, the output shape is a 4×4 matrix. So, the first element of the output is the sum of the element-wise product of the first 27 values from the input (9 values from each channel) and the 27 values from the filter. After that we convolve over the entire image.

Multiple Filters Edges

Instead of using just a single filter, we can use multiple filters as well. How do we do that? Let's say the first filter will detect vertical edges and the second filter will detect horizontal edges from the image. If we use multiple filters, the output dimension will change. So, instead of having a 4×4 output as in the above example, we would have a 4×4×2 output (if we have used 2 filters):



Generalized dimensions can be given as:

- **Input:** n×n×c
- **Filter:** f×f×c
- **Padding:** p
- **Stride:** s
- **Output:** [((n+2p-f)/s))+1] × [((n+2p-f)/s)+1]

Here, c is the number of channels in the input and filter

Once we get an output after convolving over the entire image using a filter, we add a bias term to those outputs and finally apply an activation function to generate activations. *This is one layer of a convolutional network*.

The activations from layer 1 act as the input for layer 2, and so on. Clearly, the number of parameters in case of convolutional neural networks is independent of the size of the image. It essentially depends on the filter size. Suppose we have 10 filters, each of shape 3×3×3. What will be the number of parameters in that layer? Let's try to solve this:

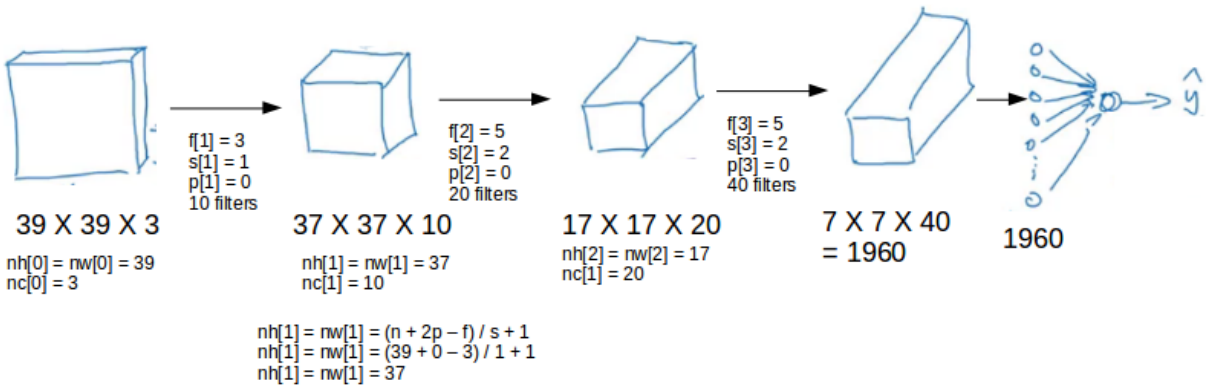
- Number of parameters for each filter = 3×3×3 = 27
- There will be a bias term for each filter, so total parameters per filter = 28
- As there are 10 filters, the total parameters for that layer = 28*10 = 280

No matter how big the image is, the parameters only depend on the filter size. Awesome, isn't it?

Let's have a look at the summary of notations for a convolution layer:

Let's combine all the concepts we have learned so far and look at a convolutional network example.

This is how a typical convolutional Neural network looks like:



We take an input image (size = 39×39×3 in our case), convolve it with 10 filters of size 3×3, and take the stride as 1 and no padding. This will give us an output of 37×37×10. We convolve this output further and get an output of 7×7×40. Finally, we take all these numbers (7×7×40 = 1960), unroll them into a large vector, and pass them to a classifier that will make predictions. This is a microcosm of how a convolutional network works.

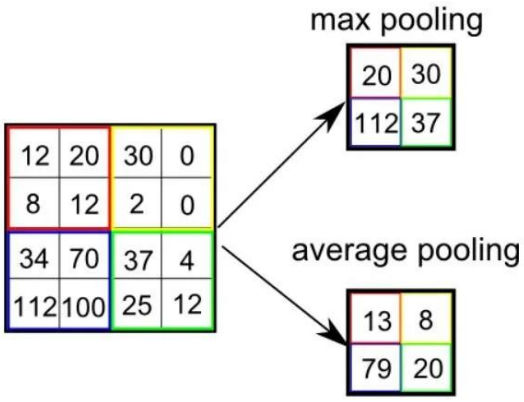
There are a number of hyperparameters that we can tweak while building a convolutional Neural networks. These include the number of filters, size of filters, stride to be used, padding, etc. As we go deeper into the network, the size of the image shrinks whereas the number of channels usually increases.

Pooling Layers

Similar to the Convolutional Layer, the Pooling layer is responsible for reducing the spatial size of the Convolved Feature. This is to **decrease the computational power required to process the data** through dimensionality reduction. Furthermore, it is useful for **extracting dominant features** which are rotational and positional invariant, thus maintaining the process of effectively training the model. There are two types of Pooling

i. **Max Pooling** returns the **maximum value** from the portion of the image covered by the Kernel.

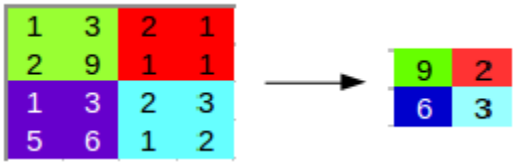
ii. **Average Pooling** returns the **average of all the values** from the portion of the image covered by the Kernel.



Pooling layers are generally used to reduce the size of the inputs and hence speed up the computation. Consider a 4×4 matrix as shown below:

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

Applying max pooling on this matrix will result in a 2×2 output:



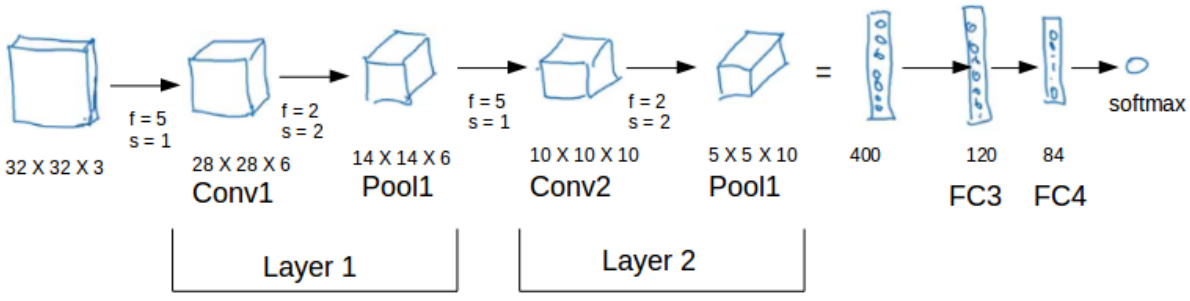
For every consecutive 2×2 block, we take the max number. Here, we have applied a filter of size 2 and a stride of 2. These are the hyperparameters for the pooling layer. Apart from max pooling, we can also apply average pooling where, instead of taking the max of the numbers, we take their average. In summary, the hyperparameters for a pooling layer are:

1. Filter size
2. Stride
3. Max or average pooling

If the input of the pooling layer is n×n×c, then the output will be [((n – f)/s + 1)×((n – f)/s + 1)× c].

Example1:

Let's look at how a convolution neural network with convolutional and pooling layer works. Suppose we have an input of shape 32×32×3:



There are a combination of convolution and pooling layers at the beginning, a few fully connected layers at the end and finally a softmax classifier to classify the input into various categories. There are a lot of hyperparameters in this network which we have to specify as well. As seen in the above example, the height and width of the input shrinks as we go deeper into the network (from 32×32 to 5×5) and the number of channels increases (from 3 to 10).

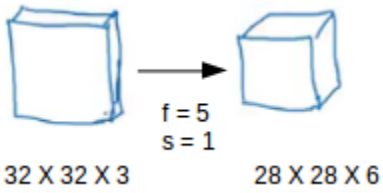
All of these concepts and techniques bring up a very fundamental question – why convolutions? Why not something else?

Why Convolutions?

There are primarily two major advantages of using convolutional layers over using just fully connected layers:

1. Parameter sharing
2. Sparsity of connections

Consider the below example:



If we would have used just the fully connected layer, the number of parameters would be = 32×32×3×28×28×6, which is nearly equal to 14 million! Makes no sense, right?

If we see the number of parameters in case of a convolutional layer, it will be = (5×5 + 1) × 6 (if there are 6 filters), which is equal to 156. Convolutional layers reduce the number of parameters and speed up the training of the model significantly.

In convolutions, we share the parameters while convolving through the input. The intuition behind this is that a feature detector, which is helpful in one part of the image, is probably also useful in another part of the image. So a single filter is convolved over the entire input and hence the parameters are shared.

The second advantage of convolution is the sparsity of connections. For each layer, each output value depends on a small number of inputs, instead of taking into account all the inputs.

Advantages of CNNs:

1. Good at detecting patterns and features in images, videos, and audio signals.
2. Robust to translation, rotation, and scaling invariance.
3. End-to-end training, no need for manual feature extraction.
4. Can handle large amounts of data and achieve high accuracy.

Disadvantages of CNNs:

1. Computationally expensive to train and require a lot of memory.

2. Can be prone to overfitting if not enough data or proper regularization is used.
3. Requires large amounts of labeled data.
4. Interpretability is limited, it's hard to understand what the network has learned.

Example2:

Consider a Convolutional Neural Network (CNN) for image classification that receives images of 84×84×1 pixels as input. The first hidden (convolutional) layer has 16 filters, filter size 7, stride 1 and pad 2. The second hidden (pooling) layer has filter size 2 and stride 2. The third hidden (convolutional) layer has 32 filters, filter size 7, stride 1 and pad 2. The fourth hidden (pooling) layer has filter size 3 and stride 3. What is number of parameters/weights in each of those layers in a model using this CNN network without fully connected layers.

SOLUTION:

1. First Convolutional Layer

- Input Size: 84×84×1
- Number of Filters: 16
- Filter Size: 7×7
- Padding: 2
- Stride: 1

Each filter will have 7×7 weights for the spatial dimensions and 1 additional weight for the bias. Therefore, the number of parameters per filter is:

(7×7×1)+1=49+1=50 parameters per filter

Since there are 16 filters so total parameters for first layer =16×50=**800**

2. First Pooling Layer

- Type: Pooling (max or average)
- Filter Size: 2×2
- Stride: 2

Pooling layers do not have parameters associated with them, so the total number of parameters for the pooling layer is 0

3. Second Convolutional Layer

After the first convolutional layer and pooling, the output size should be calculated. So the output width and height from the first convolutional layer with padding of 2 can be calculated using the formula:

Output size=(n−f+2×s+1)

Output size=(84−7+2×2+1)=82

Output size of pooling =[(n – f)/s + 1]

Output size of pooling=(82−2/2 +1)=41

The output size after the pooling is 41×41×16.

- Number of Filters: 32
- Filter Size: 7×7
- Padding: 2
- Stride: 1

(7×7×16)+1= (49×16)+1=784+1=785 parameters per filter

Since there are 32 filters:

Total parameters for second layer=32×785=25,120

4. Second Pooling Layer

- Filter Size: 3×3
- Stride: 3

Similar to the first pooling layer, pooling layers do not have parameters, so total parameters at pooling layer=0

Total Parameters in the Model

The total number of parameters in this CNN (excluding fully connected layers) is:

800+0+25,120+0 = **25,920** parameters.

CNN Case-studies:

We learned about Conv layer, pooling layer, and fully connected layers. It turns out that computer vision researchers spent the past few years on how to put these layers together. To get some intuitions you have to see the examples. These neural network architectures that works well in some tasks can also work well in other tasks.

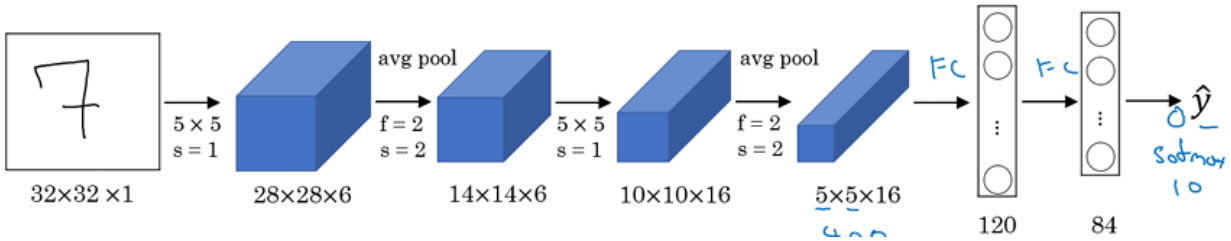
Here are some classical CNN networks which worked good for images datasets e.g. **LeNet-5**, **AlexNet**, and **VGG**. The best CNN architecture that won the last **ImageNet** competition is called

ResNet and it has 152 layers. There are also an architecture called **Inception** that was made by Google that are very useful to learn and apply to your tasks. Reading and trying the mentioned models can boost you and give you a lot of ideas to solve your task.

LeNet-5

The goal for this model was to identify handwritten digits in a 32x32x1 gray image.

This model was published in 1998.



The last layer wasn't using softmax back then. It has 60k parameters. The dimensions of the image decreases as the number of channels increases. The activation function used in the original work was Sigmoid and Tanh. Modern implementation uses RELU in most of the cases.

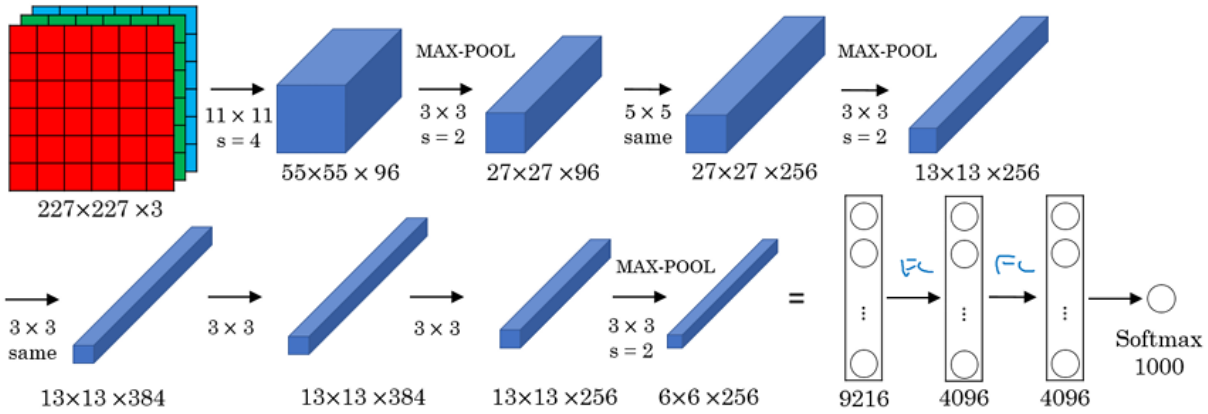
Layer	Type	Maps	Size	Kernel size	Stride	Activation
Out	Fully Connected	–	10	–	–	RBF
F6	Fully Connected	–	84	–	–	tanh
C5	Convolution	120	1 × 1	5 × 5	1	tanh
S4	Avg Pooling	16	5 × 5	2 × 2	2	tanh
C3	Convolution	16	10 × 10	5 × 5	1	tanh
S2	Avg Pooling	6	14 × 14	2 × 2	2	tanh
C1	Convolution	6	28 × 28	5 × 5	1	tanh
In	Input	1	32 × 32	–	–	–

AlexNet

It was named after Alex Krizhevsky who was the first author of this paper. The goal for the model was the **ImageNet** challenge which classifies images into **1000** classes. It won the 2012 ImageNet ILSVRC competition. It was similar to LeNet-5 but bigger than it with 60 Million parameters to be learned as compared to 60k parameter of LeNet-5. It was the first architecture

which stacked convolutional layers directly on top of each other instead of stacking a pooling layer on top of convolutional layer. It used the RELU activation function.

Here is the drawing of the model:



Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully Connected	–	1,000	–	–	–	Softmax
F9	Fully Connected	–	4,096	–	–	–	ReLU
F8	Fully Connected	–	4,096	–	–	–	ReLU
C7	Convolution	256	13 × 13	3 × 3	1	SAME	ReLU
C6	Convolution	384	13 × 13	3 × 3	1	SAME	ReLU
C5	Convolution	384	13 × 13	3 × 3	1	SAME	ReLU
S4	Max Pooling	256	13 × 13	3 × 3	2	VALID	–
C3	Convolution	256	27 × 27	5 × 5	1	SAME	ReLU
S2	Max Pooling	96	27 × 27	3 × 3	2	VALID	–
C1	Convolution	96	55 × 55	11 × 11	4	VALID	ReLU
In	Input	3 (RGB)	227 × 227	–	–	–	–

VGG-16

It was a modification of AlexNet proposed in 2015. Focus on having only these blocks: CONV = 3×3 filter, s = 1, same MAX-POOL = 2×2 , s = 2.

This network is large even by modern standards. It has around 138 million parameters. There are another version called VGG-19 which is a bigger version. But most people uses the VGG-16 instead of the VGG-19 because it does the same.

Here is the architecture: